ファイル書き込み時のページキャッシュ利用特性およびファイル置換挙動に基づく軽量なランサムウェア検知手法

河田 悠翔[1]　佐藤 将也[2]　山内 利宏[3]

概要：ファイル I/O に基づくランサムウェア検知手法では，I/O パターンの解析やエントロピ算出などが実行時オーバヘッドの要因となる．この課題に対し，監視対象プロセスや取得する情報を実行時の挙動に応じて動的に制御する手法が提案されている．しかし，不審なプロセスには依然として詳細な挙動解析を要する．本研究では，ファイル書き込み時のページキャッシュ利用特性およびファイル置換挙動に基づく軽量なランサムウェア検知手法を提案する．提案手法は，これらの挙動に対応するカーネル処理の実行回数をプロセス単位かつ一定時間単位で集計し，その集計値からランサムウェアの特徴を捉える．これにより，検知に必要な情報および解析処理を簡素化する．評価の結果，提案手法は既存ツールと同等以上の検知結果を示した．また，平均ユーザ CPU 使用率および平均カーネル CPU 使用率の増加はそれぞれ 0.02pp，0.58pp，tar 実行時間の増加率は 1.18%であり，既存ツールより小さい実行時オーバヘッドを示した．

キーワード：ランサムウェア，ページキャッシュ，ファイル I/O，Linux

A Lightweight Ransomware Detection Method Based on Page Cache Usage during File Writes and File Replacement Behavior

Haruto Kawata[1]　Masaya Sato[2]　Toshihiro Yamauchi[3]

Abstract: File I/O-based ransomware detection incurs runtime overhead from I/O pattern analysis and entropy calculation. Methods that dynamically adapt monitoring and information collection to runtime behavior have been proposed. However, suspicious processes still require detailed behavioral analysis. In this study, we propose a lightweight ransomware detection method based on page cache usage during file writes and file replacement behavior. The proposed method aggregates the execution counts of kernel operations corresponding to these behaviors on a per-process basis at fixed time intervals and captures ransomware characteristics from the aggregated values. This approach simplifies the information and analysis required for detection. Our evaluation showed that the proposed method achieved detection results at least comparable to those of existing tools. The increases in average user and kernel CPU utilization were 0.02 and 0.58 percentage points, respectively, while the increase in tar execution time was 1.18%, indicating lower runtime overhead than the existing tools.

1. はじめに

ファイルを暗号化し，その復元と引き換えに身代金を要求するランサムウェアによる被害は，高い水準で推移している [1]．ランサムウェアによる攻撃は，金銭的な被害にとどまらず，医療機関などの社会基盤にまで深刻な影響を及ぼす [2]．このため，ランサムウェアを早期に検知し，被害を抑えることは重要な課題である．

[1] 岡山大学大学院環境生命自然科学研究科
Graduate School of Environmental, Life, Natural Science and Technology, Okayama University
[2] 岡山県立大学情報工学部
Faculty of Computer Science and Systems Engineering, Okayama Prefectural University
[3] 岡山大学学術研究院環境生命自然科学学域
Faculty of Environmental, Life, Natural Science and Technology, Okayama University

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

ランサムウェア対策として，実行時の振る舞いを動的に解析する手法が提案されている．その中でも，ファイル I/O の監視を用いる手法に着目する．ファイル I/O に基づく既存手法 [3], [4], [5] では，I/O 操作の頻度やパターン，読み込みや書き込みデータのエントロピ，およびアクセス対象ファイルの情報などからランサムウェアに特徴的な挙動を捉える．しかし，これらの情報取得に伴う監視や解析処理は，実行時オーバヘッドの要因となる [6]．この課題に対し，監視対象とするプロセスや取得する情報を実行時の挙動に応じて動的に制御し，オーバヘッドを低減する手法が提案されている [6], [7]．しかし，これらの手法においても，不審なプロセスに対してはエントロピ算出や I/O パターンの解析が必要となる．このため，監視対象を限定するだけでなく，検知に必要な情報および解析処理を簡素化することで，さらなる軽量化を図る余地がある．

我々はこれまで，ページキャッシュヒット率と先読みページ数に着目し，ランサムウェアと良性ソフトウェアの挙動差異を分析した [8]．この結果，一部のランサムウェアでは差異が確認されたものの，評価対象としたランサムウェアに共通する傾向は確認されなかった．そこで，本研究では，ページキャッシュの監視をファイル書き込み時に限定し，その利用特性とファイル置換挙動に基づく軽量なランサムウェア検知手法を提案する．ランサムウェアの暗号化処理では，対象ファイルの更新方法に応じて，ページキャッシュの利用，ファイルの作成と削除，およびファイル時刻情報の設定操作に特徴的な挙動が現れる．提案手法では，これらの挙動に対応するカーネル処理の実行回数をプロセス単位かつ一定時間単位で集計し，その集計値からランサムウェアの特徴を捉える．

評価では，提案手法を Linux 上に実装し，ソフトウェア単独実行時および共存実行時の検知性能を評価した．また，Linux で動作する既存の検知ツールを同一の評価環境で実行し，検知性能および実行時オーバヘッドを比較した．

2. ランサムウェア検知の背景と課題

2.1 ランサムウェアによる暗号化処理

ランサムウェアは，暗号化型とロッカー型に大別される [9]．暗号化型は，ファイルを暗号化し，その復元と引き換えに身代金を要求する．本稿では暗号化型を対象とし，以降ではこれをランサムウェアと表記する．

ランサムウェアは，暗号化処理の挙動に基づき，in-place 型と out-of-place 型に大別される [10]．in-place 型は，対象ファイルを読み込み，生成した暗号化データを同一ファイルへ書き込む．out-of-place 型は，対象ファイルとは別にファイルを作成して暗号化データを書き込み，元ファイルを削除する．

本研究では，攻撃者が Linux 上のアカウントを侵害し，その権限でランサムウェアを実行する状況を想定する．ランサムウェアは，侵害されたアカウントがアクセス可能なファイルに対して，読み込み，書き込み，作成，および削除を行う．また，攻撃者はカーネルおよび提案手法を停止または改変する権限を有さないものとする．

2.2 ファイル I/O に基づく検知手法とオーバヘッド

ランサムウェアは暗号化の過程で特徴的なファイル I/O を発生させる．この性質を利用し，ファイル I/O を監視してランサムウェアを検知する手法が提案されている．これらの手法では，主に以下の特徴が利用される [6]．

- **I/O 操作の頻度とパターン：**読み込み，書き込み，作成，名前変更，および削除等の発生頻度や操作パターンを利用する．
- **データのエントロピ：**暗号化データが高いエントロピを示す性質を利用する．
- **アクセス対象のファイル形式とディレクトリ走査：**特定のファイル形式を対象とし，多数のディレクトリを走査して暗号化対象を探索する挙動を利用する．

上記の特徴を取得および解析する処理は，実行時オーバヘッドの要因となる [6]．多数の I/O 操作の継続的な監視と記録，データのエントロピ算出，およびファイル名解決などのカーネルの追加処理が必要となるためである．

この課題に対し，プロセスの挙動に応じて監視対象や取得する情報を動的に制御する手法が提案されている [6], [7]．しかし，不審と判定されたプロセスに対しては，エントロピ算出や I/O パターン解析が依然として必要となる．このため，監視対象を限定するだけでなく，検知に必要な情報および解析処理を簡素化することで，さらなる軽量化を図る余地がある．

2.3 ページキャッシュを利用したファイル I/O

ページキャッシュとは，外部記憶装置との入出力処理のデータをメモリに保持することで，外部記憶装置へのアクセス回数を削減する機構である．Linux をはじめとする多くのオペレーティングシステムでは，ページキャッシュはディスク入出力の効率化を目的として利用されている．Linux では，ページキャッシュ上のファイルデータを folio と呼ばれる単位で管理する．

ページキャッシュを利用したファイル書き込み処理の流れを説明する．プロセスがファイルに対して書き込み要求を発行すると，カーネルは書き込み対象位置に対応する folio をページキャッシュから検索する．検索の結果，対象 folio がページキャッシュ上に存在する場合（キャッシュヒット），当該 folio を取得し，ユーザ空間のバッファ領域から書き込みデータをコピーする．存在しない場合（キャッシュミス）には，新たな folio を確保してページキャッシュへ追加した後，書き込みデータをコピーする．

© 2026 Information Processing Society of Japan

3. 提案手法の検知アプローチ

3.1 検知指標の設計方針

本研究では，ランサムウェアに特徴的なファイル I/O 挙動について，カーネル処理の実行回数を集計することで捉える．これにより，検知に必要な情報および解析処理の簡素化を図る．本方針に基づき，in-place 型と out-of-place 型の暗号化処理の特徴から，ファイル書き込み時のページキャッシュ利用特性とファイル置換挙動に着目する．

3.1.1 ファイル書き込み時のページキャッシュ利用特性

in-place 型は，暗号化データの書き込み時，先行する読み込み処理によって対象データがページキャッシュ上に格納されている可能性が高い．そこで，本特性を捉えるため，書き込み処理におけるページキャッシュヒット率（Page Cache Hit Ratio：PCHR）に着目する．本研究における PCHR とは，ファイル I/O に伴って対象 folio をページキャッシュから検索した回数のうち，キャッシュヒットとなった回数の割合である．また，ランサムウェアは，多数のファイルを短時間に暗号化するため，ファイルに対する読み書きを高頻度に発生させる [6]．そこで，ページキャッシュアクセス回数にも着目することで，キャッシュヒットの割合と利用頻度の両面からランサムウェアの挙動を捉える．

3.1.2 ファイル置換挙動

out-of-place 型は，新規ファイルの作成と元ファイルの削除を対象ファイルごとに繰り返す．このため，ファイルの作成数と削除数には一定の対応関係が生じると考えられる．そこで，本特性を捉えるため，ファイル作成数と削除数の比率に着目する．また，ランサムウェアの主目的はファイルを利用不能にすることであり [5]，処理後のファイルに元ファイルの時刻情報を反映することは，暗号化処理の成立に必須ではないと考えられる．このため，ファイル時刻情報の設定操作にも着目する．

3.2 指標の定義と算出方法

3.2.1 ファイル書き込み時のページキャッシュ利用特性

書き込み処理における PCHR は，書き込み処理に伴うページキャッシュの検索回数（L_WRITE），およびキャッシュミス発生回数（M_WRITE）から近似的に算出する．これは，プロセス単位の PCHR がカーネルによって直接管理されていないためである．計算式を以下に示す．

$$\frac{(\text{L_WRITE} - \text{M_WRITE})}{\text{L_WRITE}} \times 100 \qquad (1)$$

分子 L_WRITE − M_WRITE は，キャッシュヒット回数を近似的に表す．なお，L_WRITE の値が 0 の場合は，PCHR の値を 0 とする．

各項の値の取得のために以下のカーネル関数を監視する．

- **L_WRITE：__filemap_get_folio() の呼び出し回数**
- **M_WRITE：filemap_add_folio() の呼び出し回数**

`__filemap_get_folio()` は，指定されたファイル位置に対応する folio をページキャッシュから検索して取得する関数である．本研究では，当該関数の呼び出し回数をページキャッシュの検索回数として扱う．また，ページキャッシュアクセス回数には，この検索回数を用いる．`filemap_add_folio()` は，ページキャッシュ上に対象 folio が存在せず，新たに確保した folio をページキャッシュへ追加する際に呼び出される関数である．本研究では，当該関数の呼び出し回数をキャッシュミス回数として扱う．また，監視対象を書き込み処理に限定するため，`ksys_write()` の実行期間を書き込み処理中と定義し，当該期間内に呼び出された関数のみを監視対象とする．

3.2.2 ファイル置換挙動

ファイル作成数と削除数の比率：新規ファイルの作成時に呼び出される `ext4_create()`，およびファイル削除時に呼び出される `ext4_unlink()` の呼び出し回数を用いて次式で定義する．

$$\frac{\text{ext4_unlink() の呼び出し回数}}{\text{ext4_create() の呼び出し回数}} \qquad (2)$$

`ext4_create()` の呼び出し回数が 0 の場合は，比率を 0 とする．また，新規作成された一時ファイル等の削除を元ファイルの削除として計数することを避けるため，削除対象ファイルの作成から削除までの経過時間を用いる．本研究では，この経過時間が 60 秒以上である場合に元ファイル候補として計数する．経過時間は，削除対象ファイルの作成時刻と現在時刻の近似値との差分から算出する．

ファイル時刻情報の設定操作：ファイル時刻情報を設定する際に呼び出される `do_utimes()` を監視し，呼び出し回数を計数する．

3.3 指標の妥当性調査

前節で定義した指標について，ランサムウェアと良性ソフトウェアの判別に有効な差異が現れるかを分析した．各ソフトウェアを単独で実行し，1 秒間隔で各指標を算出した．良性ソフトウェアには，書き込み方式やファイルの作成および削除方式が異なるものを選定した．

3.3.1 in-place 型に対する検証

分析対象とした 5 種の in-place 型はいずれも，PCHR が 80%以上となる測定区間が全体の 8 割以上を占めた．この結果より，書き込み時の高い PCHR は，分析対象の in-place 型に共通する傾向として確認された．

一方，良性ソフトウェアでは，システムログの記録処理，およびファイルを上書きする設定で実行した暗号化処理でも高い PCHR が観測された．前者では，ページキャッシュアクセス回数は最大 26 回/秒であった．これに対し，in-place 型では中央値が 56〜3,322 回/秒であり，5 種中 4

© 2026 Information Processing Society of Japan

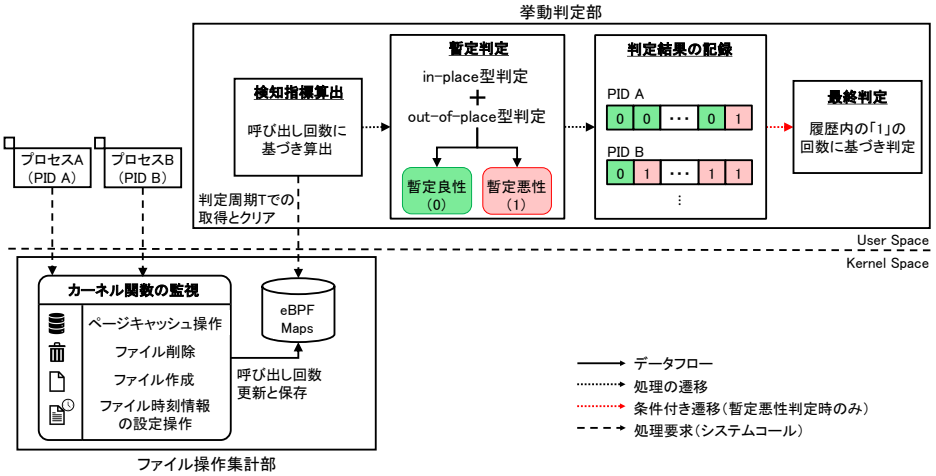


図 1　提案手法の全体像

種では 1,000 回/秒を超えた．後者では，ファイル時刻情報の設定操作が測定区間の 80%以上で観測されたのに対し，in-place 型では観測されなかった．

以上より，PCHR に加えてページキャッシュアクセス回数およびファイル時刻情報の設定操作を用いることで，良性ソフトウェアとの判別が可能であることを確認した．

3.3.2　out-of-place 型に対する検証

分析対象とした out-of-place 型では，ファイル作成数と削除数の比率が 0.9 以上 1.1 以下となる測定区間が全体の 94.5%を占めた．この結果から，分析対象の out-of-place 型では，ファイル作成数と削除数が同程度の値を示す傾向を確認した．

一方，圧縮や暗号化といった既存データの形式変換を行う良性ソフトウェアでも同様の傾向が確認された．当該ソフトウェアでは，ファイル時刻情報の設定操作が測定区間の 88%以上で観測されたのに対し，分析対象の out-of-place 型では観測されなかった．

以上より，ファイル作成数と削除数の比率およびファイル時刻情報の設定操作を用いることで，良性ソフトウェアとの判別が可能であることを確認した．

この検証結果に基づき，本研究では，書き込み処理における PCHR，ページキャッシュアクセス回数，ファイル作成数と削除数の比率，およびファイル時刻情報の設定操作をランサムウェアの検知指標として使用する．

4.　提案手法

4.1　設計方針

提案手法は，監視に伴うシステム負荷の低減および誤検知の抑制を目的として，以下の 2 つの設計方針を定める．

集計値の周期的取得による監視負荷の低減：検知指標の算出に用いる集計値をユーザ空間から一定周期ごとに一括して取得する．これにより，データ取得回数をカーネル関数の呼び出し頻度に依存させず，監視に伴う負荷を低減する．

一定期間の判定結果に基づく誤検知の抑制：各周期で得られた判定結果を一定期間保持し，複数周期の結果に基づいて最終判定を行う．単一周期の結果のみで判定した場合，良性ソフトウェアが一時的にランサムウェアと類似した挙動を示すことで，誤検知が生じる可能性がある．そこで，一定期間の結果を総合して判定することで，一時的な類似挙動が判定に与える影響を抑え，継続的に不審な挙動を示すプロセスを検知する．

4.2　全体像

提案手法の全体像を**図 1** に示す．提案手法では，eBPF を用いて指標算出に必要なカーネル関数を監視し，各関数の呼び出し回数をプロセス単位で計数する．ユーザ空間では，計数した呼び出し回数に基づき，検知指標を算出して判定を行う．

4.2.1　ファイル操作集計部

ファイル操作集計部は，2 つの処理から構成される．

- **第 1 段階 カーネル関数の呼び出し監視：**監視対象のカーネル関数をフックし，各関数の呼び出しを捕捉する．
- **第 2 段階 データ取得および格納：**関数呼び出し時，呼び出し元のプロセス ID や引数情報を取得する．これらの情報に基づき，eBPF Maps に保持した当該プロセスに対応する関数の呼び出し回数を更新する．

4.2.2　挙動判定部

挙動判定部は，4 つの処理から構成される．

- **第 1 段階 検知指標の算出：**判定周期 T ごとにカーネル空間から情報を取得し，検知指標を算出する．
- **第 2 段階 暫定判定処理：**算出した指標に基づき，in-place 型および out-of-place 型の挙動を判別する．いずれかの判定条件を満たしたプロセスを暫定悪性 (1)，満たさなかったプロセスは暫定良性 (0) として分類する．

© 2026 Information Processing Society of Japan

- **第 3 段階 判定結果の記録：**各判定周期の暫定判定結果は，最大 N 周期分の履歴としてプロセス単位で保持する．新たな判定結果の追加により，保持数が N を超える場合は，最も古い判定結果を捨てる．
- **第 4 段階 最終判定処理：**暫定悪性判定時，履歴内の暫定悪性（1）の回数に基づき，対象プロセスを良性，不審，悪性のいずれかに分類する．

4.3 判定条件とパラメータ設定

提案手法では，判定周期，判定履歴長，暫定判定条件，および最終判定条件を以下のように設定する．

判定周期と判定履歴長：判定周期 T を 1 秒，判定履歴長 N を 10 とする．判定周期は，プロセスの挙動を 1 秒単位で分析する既存研究 [7] を参考に設定した．また，履歴長は 10 秒間の I/O 挙動に基づいてランサムウェアを判定する既存研究 [11] を参考に設定した．

暫定判定条件：3.3 節で確認した指標値の傾向に基づき，以下のいずれかを満たす場合に暫定悪性と判定する．

- in-place 型：PCHR が 80%以上，かつページキャッシュアクセス回数が 30 回以上，かつファイル時刻情報の設定操作が観測されない場合
- out-of-place 型：ファイル作成数と削除数の比率が 0.9 以上 1.1 以下，かつファイル時刻情報の設定操作が観測されない場合

最終判定条件：直近 10 回の暫定判定結果のうち，暫定悪性と判定された回数を K とする．$K \leq 5$ を良性，$6 \leq K \leq 7$ を不審，$K \geq 8$ を悪性と判定する．判定履歴中の暫定悪性が過半数となる 6 回以上を不審とし，より高い継続性を示す 8 回以上を悪性とした．

5. 評価

5.1 評価方針

評価では，提案手法の検知性能およびシステム性能への影響を明らかにするため，以下の 3 つの Research Question（RQ）を設定する．

- **RQ1：**ソフトウェアを単独で実行した場合，提案手法は良性ソフトウェアとランサムウェアをどの程度正確に判別できるか．
- **RQ2：**複数のソフトウェアが共存して動作する環境において，提案手法は良性ソフトウェアとランサムウェアをどの程度正確に判別できるか．
- **RQ3：**提案手法の検知性能および実行時オーバヘッドは，既存ツールと比較してどの程度か．

5.2 評価対象ソフトウェアと評価環境

5.2.1 評価対象ソフトウェア

評価に使用したソフトウェアを**表 1** に示す．本評価では，ランサムウェア 8 種および良性ソフトウェア 9 種を対象とした．ランサムウェアは，3.3 節で用いた 6 種に AvosLocker と Monti を加えた．このうち，7 種が in-place 型，1 種が out-of-place 型である．これらの検体は，MalwareBazaar [12] から取得した．

表 1 評価に用いたソフトウェア

分類	ソフトウェア
in-place 型	IceFire, Nitrogen, RansomEXX, INC, HelloKitty，AvosLocker，Monti
out-of-place 型	SougoLock
良性ソフトウェア	gzip（圧縮），tar（展開），rsync（同期），cp（コピー），ccrypt（暗号化），tar（アーカイブ），find + shred（セキュア削除），sqlite3（データベース），make（ビルド）

表 2 評価環境

ホスト OS	Ubuntu 24.04.2 LTS
CPU	AMD Ryzen 7 5800X 8-Core Processor
メモリ	128 GB
ストレージデバイス	HDD
仮想化ソフトウェア	VirtualBox 7.1.8
ゲスト OS	Ubuntu 24.04.1 LTS
カーネル	Linux v6.8.12
メモリ	6 GB
コア数	4

良性ソフトウェアには，3.3 節で分析対象としたものを一部含め，実運用環境で利用され，複数のファイルに対して I/O 処理を実行するものを選定した．これにより，多様なファイル I/O 処理に対する提案手法の誤検知の有無を評価した．

5.2.2 評価環境

評価環境を**表 2** に示す．各ソフトウェアは，VirtualBox 上の Ubuntu で実行した．本環境はランサムウェアによる攻撃がネットワーク経由で他計算機に影響を及ぼさないために，外部ネットワークから隔離した．

sqlite3 および make を除くソフトウェアの処理対象には，公開データセット NapierOne [13] のサブセットである NapierOne-tiny から選定した実ファイルを使用した．これにより，多様な実ファイルを処理する際のソフトウェアの挙動を評価する．本評価では，ランサムウェアの標的となる 44 種類のファイル形式（例：PDF，PPTX 等）から各 100 ファイルずつ，計 4,400 ファイル（約 6 GB）を使用した．make については，Redis のソースコードを対象としてビルド処理を実行した．また，sqlite3 については，SQLite 開発チームによる性能評価用スクリプトから生成した SQL 文を用いた．これは，10 万件のデータに対して数十万回規模の検索や更新処理，数万件規模の削除処理，インデックス作成，およびデータベース再編成を実行するものである．

5.3 RQ1：単独実行時の検知性能

提案手法を起動して 10 秒経過した後，各ソフトウェア

© 2026 Information Processing Society of Japan

表 3　各手法におけるランサムウェアの検知結果

ランサムウェア	提案手法	Rekd	TiamatAV
IceFire	✓	✓	✓
Nitrogen	✓	✓	✓
RansomEXX	✓	✓	✓
INC	✓	✓	✓
HelloKitty	✓	✓	✓
AvosLocker	✓	✓	✓
Monti	✓	✓	✓
SougoLock	✓	✓	×
検知成功数	**8/8**	**8/8**	**7/8**

✓：検知成功，×：検知失敗

表 4　各手法における良性ソフトウェアの誤検知の評価結果

良性ソフトウェア	提案手法	Rekd	TiamatAV
gzip	✓	×	✓
tar（展開）	✓	×	✓
rsync	✓	×	✓
cp	✓	✓	✓
ccrypt	✓	×	✓
tar（アーカイブ）	✓	×	✓
find + shred	✓	×	✓
sqlite3	✓	✓	✓
make	✓	✓	✓
誤検知数	**0/9**	**6/9**	**0/9**

✓：誤検知なし，×：誤検知あり

を単独で 1 回ずつ実行した．検知結果は，提案手法が出力する判定ログに基づいて評価した．ランサムウェアについては，実行したランサムウェアのプロセスに対して悪性判定が出力された場合を検知成功とした．良性ソフトウェアについては，実行した良性ソフトウェアのプロセスに対して悪性判定が出力された場合を誤検知とした．

表 3 に，提案手法によるランサムウェア 8 種の検知結果を示す．提案手法は，評価対象としたランサムウェア 8 種すべてを検知した．また，**表 4** に，良性ソフトウェア 9 種の誤検知の評価結果を示す．提案手法は，評価対象とした良性ソフトウェア 9 種のいずれについても誤検知は確認されなかった．

5.4　RQ2：共存実行時の検知性能

本評価では，良性ソフトウェア同士が共存して動作する場合，および 1 種のランサムウェアと 2 種の良性ソフトウェアが共存して動作する場合を対象とした．良性ソフトウェアの組合せには，処理特性の異なる以下の 2 条件を設定した．

- 条件 A：gzip + ccrypt
- 条件 B：rsync + tar（展開）

条件 A では，ファイル内容の変更，ファイル作成，およびファイル削除といったランサムウェアと類似したファイル I/O 挙動を示す良性ソフトウェアを実行した．条件 B では，データ複製やアーカイブ展開といった汎用的なファイル I/O 処理を行う良性ソフトウェアを実行した．各ソフトウェアには，5.2.2 項で述べたデータセットを用意し，互いに重複しないファイル群を処理対象として割り当てた．これにより，同一ファイルに対する競合の影響を排除しつつ，同一ファイルシステム上で複数のファイル I/O が共存して発生する環境を構成した．ランサムウェアを含む条件では，2 種の良性ソフトウェアを起動した 10 秒後にランサムウェアを実行した．良性ソフトウェア同士の共存実行は各条件について 1 回，ランサムウェアを含む共存実行は各ランサムウェアと各条件の組合せについて 1 回ずつ実施した．検知および誤検知の判定基準は 5.3 節と同様である．

表 5　良性ソフトウェアの共存実行時における誤検知の評価結果

条件	共存実行した良性ソフトウェア	誤検知
A	gzip + ccrypt	✓
B	rsync + tar（展開）	✓

✓：誤検知なし，×：誤検知あり

表 6　ランサムウェアと良性ソフトウェアの共存実行時の評価結果

	条件 A		条件 B	
ランサムウェア	検知	良性誤検知	検知	良性誤検知
IceFire	✓	✓	×	✓
Nitrogen	✓	✓	✓	✓
RansomEXX	✓	✓	✓	✓
INC	✓	✓	✓	✓
HelloKitty	✓	✓	✓	✓
AvosLocker	✓	✓	✓	✓
Monti	✓	✓	✓	✓
SougoLock	✓	×（ccrypt）	✓	✓
結果集計	**8/8**	**1/8**	**7/8**	**0/8**

検知：✓ 検知成功，× 検知失敗

良性誤検知：✓ なし，× あり（括弧内は誤検知したソフトウェア）

良性ソフトウェア同士の共存実行時における誤検知の評価結果を**表 5** に示す．条件 A および条件 B のいずれでも誤検知は確認されなかった．また，ランサムウェアと良性ソフトウェアの共存実行時の結果を**表 6** に示す．条件 A では，評価対象とした 8 種のランサムウェアをすべて検知したものの，SougoLock との共存実行時に ccrypt を誤検知した．条件 B では，良性ソフトウェアに対する誤検知は発生しなかったものの，IceFire に対する見逃しが発生した．

5.5　RQ3：既存ツールとの比較

5.5.1　比較対象の既存ツール

比較対象には，Linux 上で動作し，ソースコードが公開されており，ファイル I/O に基づいて実行中のプロセスをランサムウェアと判定する既存ツールを選定した．さらに，提案手法と同一のソフトウェアを使用し，検知性能と実行時オーバヘッドを計測可能であることを条件とした．これらの条件を満たす既存ツールとして，Rekd [14] および TiamatAV [15] を比較対象とした．

© 2026 Information Processing Society of Japan

表 7　計測条件別の tar による展開処理実行時の CPU 使用率および実行時間の比較結果

計測条件	平均ユーザ CPU 使用率（%）	Baseline との差分（pp）	平均カーネル CPU 使用率（%）	Baseline との差分（pp）	tar 実行時間（s）	実行時間増加率（%）
Baseline	0.00 ± 0.00	N/A	1.19 ± 0.68	N/A	97.83 ± 1.83	N/A
提案手法	0.02 ± 0.02	+0.02	1.77 ± 0.84	+0.58	98.98 ± 1.89	+1.18
Rekd	0.47 ± 0.03	+0.47	3.66 ± 0.30	+2.47	99.89 ± 3.83	+2.11
TiamatAV	9.31 ± 0.36	+9.31	6.20 ± 0.39	+5.01	140.78 ± 3.69	+43.90

平均 CPU 使用率，tar 実行時間：平均値 ± 標準偏差

Rekd は，ファイルへの書き込みデータの Shannon エントロピを算出し，プロセス ID ごとの総書き込み量および高エントロピ書き込み量に基づいてランサムウェアを判定する．本評価では，Rekd は変更を加えずに使用した．

TiamatAV は，Tetragon が取得したプロセス情報およびファイル操作情報に 12 種類の検知規則を適用し，プロセス単位で集計した脅威スコアに基づいて判定する．検知規則では，ファイル更新，削除，拡張子の変更，および高エントロピなファイル書き込みなどを検出する．本評価では，処理対象ディレクトリに対するリネーム操作を取得するため，Tetragon の Tracing Policy に当該ディレクトリを追加した．この設定変更を除き，TiamatAV には変更を加えていない．

5.5.2　比較方法

各手法を起動した後に評価対象ソフトウェアを単独で 1 回ずつ実行し，各手法が出力する判定ログから検知結果を評価した．検知および誤検知の判定基準は 5.3 節と同様である．

実行時オーバヘッドの比較では，各手法を動作させない Baseline，提案手法，Rekd，および TiamatAV をそれぞれ動作させた 4 条件を評価した．ワークロードには，多数のファイル作成および読み書きを発生させる tar のアーカイブ展開処理を用いた．処理対象には，NapierOne に含まれる TXT 形式のファイル 5,000 個（4,160 MB）を格納した tar アーカイブを使用した．

評価指標には，監視および検知処理の計算負荷を評価するため，平均ユーザ CPU 使用率と平均カーネル CPU 使用率を用いた．また，アプリケーションの実行性能への影響を評価するため，tar の実行時間を用いた．CPU 使用率は `vmstat` を用いて 1 秒間隔で取得し，実行時間は`/usr/bin/time` を用いて計測した．各条件について 5 回計測し，その平均値を比較した．

5.5.3　比較結果

表 3 および表 4 に，各手法の検知結果を示す．また，**表 7** に条件別の平均 CPU 使用率および tar 実行時間を示す．提案手法は，評価対象としたランサムウェアを Rekd と同様にすべて検知し，良性ソフトウェアでは TiamatAV と同様に誤検知が確認されなかった．また，平均 CPU 使用率および tar 実行時間の増加は，Rekd および TiamatAV のいずれよりも小さかった．以上より，評価した条件において，提案手法は既存ツールと同等以上の検知結果を示し，より小さい実行時オーバヘッドで動作することを確認した．

6.　議論

6.1　共存実行が判定結果に与える影響

RQ2 の評価では，条件 B で IceFire の見逃し，条件 A では ccrypt の誤検知が発生した．IceFire では，ページキャッシュアクセス回数が閾値である 30 回を下回る区間が生じた．閾値を 15 回に変更した追加検証では検知されたことから，共存実行により単位時間当たりの処理量が低下し，閾値を満たさなくなった可能性がある．

ccrypt では，PCHR が 100%に近い値を示すものの，一部の区間でファイル時刻情報の設定操作が観測されなかった．判定周期を 5 秒に変更した追加検証では，誤検知は発生しなかった．この結果から，共存実行時には当該操作の発生が一部の区間に偏った可能性が示唆される．このため，1 秒周期では当該操作が観測されない区間が生じ，誤検知につながったと考えられる．

以上より，共存実行時には単位時間当たりの処理量や処理の発生状況が変化し，判定周期や閾値の設定によっては，見逃しや誤検知が生じることを確認した．

6.2　提案手法の適用範囲と限界

本手法は，暗号化処理に伴うファイル I/O の特徴に基づくため，攻撃者がこれらの特徴を意図的に変化させた場合，検知が遅延または回避される可能性がある．例えば，暗号化頻度を低下させることで，単位時間当たりのファイル I/O が減少する．この結果，判定条件を満たす回数が低下し，検知遅延や見逃しにつながる可能性がある．しかし，暗号化処理に要する時間を増加させるため，検知回避と暗号化頻度の間にはトレードオフが生じると考えられる．また，本手法は検知指標をプロセス単位で算出するため，暗号化処理を複数プロセスに分散した場合，見逃しが発生する可能性がある．特に out-of-place 型では，ファイル作成と削除を異なるプロセスが実行した場合，ファイルの作成数と削除数の対応関係を捉えられない．さらに，in-place 型の検知はページキャッシュ利用特性に基づき，現実装では `ksys_write()` の実行中に呼び出されるカーネル関数のみを監視する．このため，Direct I/O および `ksys_write()` を経由しない書き込みに対する検知性能は未検証である．

© 2026 Information Processing Society of Japan

現実装ではファイル作成と削除の監視に ext4 固有の関数を用いるため，適用対象は ext4 に限定される．しかし，他のファイルシステムにおいても，ファイル作成と削除に対応する処理を監視することで，本手法を適用可能であると考えられる．

本評価は，8 種のランサムウェアと 9 種の良性ソフトウェアを対象とし，HDD を用いた単一の評価環境で実施した．このため，評価対象外のソフトウェアや SSD など I/O 特性の異なる環境において，同様の検知性能が得られるとは限らない．

7. 関連研究

ファイル I/O に基づくランサムウェア検知では，I/O 操作の頻度やパターン，アクセス対象ファイルの種別，および書き込みデータのエントロピなどが利用される [3], [4]．しかし，これらの情報の取得や解析は，実行時オーバヘッドの要因となることが指摘されている [6]．

この課題に対し，監視対象や取得する情報を動的に制御し，オーバヘッドを低減する手法が提案されている．Wang らの CanCal は，デコイファイルおよびランサムノートの監視によって不審なプロセスを選別し，詳細な挙動解析を適用する [7]．また，van Sloun らは，プロセスの挙動に応じて監視段階を切り替える多段階方式を提案している [6]．良性とみられるプロセスでは取得する情報を抑え，不審なプロセスに対してのみ段階的に取得する情報を拡張する．

これらの手法が監視対象や取得する情報を動的に制御することで軽量化を実現するのに対し，本研究では，ランサムウェアの特徴的な挙動をカーネル処理の実行回数から捉える．これにより，検知に用いる情報および解析処理を簡素化し，軽量化を実現する．

8. おわりに

本稿では，検知に伴う監視および解析処理の軽量化を目的として，ファイル書き込み時のページキャッシュ利用特性とファイル置換挙動に基づく検知手法を提案し，その有効性を評価した．提案手法は，これらの挙動に対応するカーネル処理の実行回数をプロセス単位かつ一定時間単位で集計し，その集計値に基づいて検知する．

評価の結果，ソフトウェア単独実行時には 8 種のランサムウェアをすべて検知し，9 種の良性ソフトウェアでは誤検知が確認されなかった．共存実行時には，一部の条件で 1 種の見逃しと 1 種の誤検知が確認された．既存ツールとの比較では，提案手法は Rekd および TiamatAV と同等以上の検知結果を示した．また，平均カーネル CPU 使用率の増加は 0.58 pp，tar 実行時間の増加率は 1.18%であり，両ツールより実行時オーバヘッドが小さいことを確認した．

謝辞 本研究の一部は，JSPS 科研費 25K03119 および 26K14778 による助成を受けたものです．

参考文献

[1] 警察庁：令和 7 年におけるサイバー空間をめぐる脅威の情勢等について，https://www.npa.go.jp/news/release/2026/20260310001.html (accessed 2026-08-13).

[2] つるぎ町立半田病院：徳島県つるぎ町立半田病院コンピュータウイルス感染事案 有識者会議調査報告書, https://www.handa-hospital.jp/topics/2022/0616/report_01.pdf (accessed 2026-08-12).

[3] Continella, A., Guagnelli, A., Zingaro, G. et al.: ShieldFS: A Self-healing, Ransomware-aware Filesystem, *Proceedings of the 32nd annual conference on computer security applications*, pp. 336–347 (2016).

[4] Scaife, N., Carter, H., Traynor, P. et al.: CryptoLock (and Drop It): Stopping Ransomware Attacks on User Data, *2016 IEEE 36th international conference on distributed computing systems (ICDCS)*, IEEE, pp. 303–312 (2016).

[5] Kharaz, A., Arshad, S., Mulliner, C. et al.: UNVEIL: A Large-Scale, Automated Approach to Detecting Ransomware, *25th USENIX security symposium (USENIX Security 16)*, pp. 757–772 (2016).

[6] van Sloun, C., Woeste, V., Wolsing, K. et al.: Detecting Ransomware Despite I/O Overhead: A Practical Multi-Staged Approach, *NDSS* (2025).

[7] Wang, S., Dong, F., Yang, H. et al.: CanCal: Towards Real-time and Lightweight Ransomware Detection and Response in Industrial Environments, *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pp. 2326–2340 (2024).

[8] 河田悠翔，佐藤将也，山内利宏：暗号化ランサムウェア実行時のページキャッシュヒット率と先読みページ数の分析，信学技報， Vol. 125, No. 223, pp. 78–85 (2025).

[9] Oz, H., Aris, A., Levi, A. et al.: A Survey on Ransomware: Evolution, Taxonomy, and Defense Solutions, *ACM Computing Surveys (CSUR)*, Vol. 54, No. 11s, pp. 1–37 (2022).

[10] Baek, S., Jung, Y., Mohaisen, D. et al.: SSD-Assisted Ransomware Detection and Data Recovery Techniques, *IEEE Transactions on Computers*, Vol. 70, No. 10, pp. 1762–1776 (2020).

[11] Mir, W. H., Goel, N. and Tavva, V. K.: CARDR: DRAM Cache Assisted Ransomware Detection and Recovery in SSDs, *Proceedings of the International Symposium on Memory Systems*, pp. 104–115 (2024).

[12] MalwareBazaar: MalwareBazaar — Malware sample exchange, https://bazaar.abuse.ch/ (accessed 2026-08-12).

[13] NapierOne: NapierOne. A Publicly Available Modern Mixed File Data Set, http://napierone.com/Website/index.html (accessed 2026-07-28).

[14] SpiderNitt: Rekd (Ransomware Encryption Kernel Detector), https://github.com/SpiderNitt/rekd (accessed 2026-07-28).

[15] Davimalu: Detecting Ransomware on the Linux Desktop: A behavioral approach to Host-based Intrusion Detection - Source Code, https://github.com/Davimalu/detecting-ransomware-on-linux-desktop-source-code (accessed 2026-07-28).

© 2026 Information Processing Society of Japan